Projet d'Assembleur NASM

Système de Gestion de Personnel

Rapport de projet

Auteurs :

Lahcen Mokhtar AMROUNE
Mohammed Rami BENSMICHA

13 mars 2026

Table des matières

Résumé **3**

1 Introduction **4**
1.1 Contexte du projet 4
1.2 Objectifs 4
1.3 Fonctionnalités implémentées 4

2 Architecture du programme **5**
2.1 Structure du programme 5
2.1.1 Section des données (.data) 5
2.1.2 Section du code (.text) 5

3 Description détaillée du code **5**
3.1 Section .data 5
3.2 Section .bss (implicite) 6
3.3 Fonction display_menu 6
3.4 Fonction parse_input 6
3.5 Gestion de la suppression 7

4 Structures de données **8**

5 Gestion des erreurs **8**
5.1 Validation des entrées 8
5.2 Gestion des limites 8
5.3 Messages d'erreur 8

6 Exemples d'exécution **9**
6.1 Ajout d'une personne 9
6.2 Affichage de la liste 9
6.3 Calcul de la moyenne 9

7 Tests et validation **9**
7.1 Plan de tests 9
7.2 Résultats des tests 9

8 Difficultés rencontrées et solutions **10**
8.1 Difficultés techniques 10
8.2 Solutions apportées 10

9 Améliorations possibles **10**
9.1 Améliorations fonctionnelles 10
9.2 Améliorations techniques 11
9.3 Exemple d'amélioration : Macro pour sys_write 11

10 Compilation et exécution **11**
10.1 Commandes de compilation 11
10.2 Prérequis 11

11 Conclusion **12**

Résumé

Ce rapport présente le développement d'un système de gestion de personnel en langage assembleur NASM pour architecture x86 sous Linux. L'application propose une interface en ligne de commande avec un menu interactif permettant d'enregistrer, lister, supprimer des personnes, ainsi que de calculer des statistiques démographiques. Ce projet démontre une maîtrise approfondie de la programmation bas niveau, de la gestion mémoire et des appels système.

Mots-clés : Assembleur NASM, x86, Linux, gestion de données, appels système, programmation bas niveau

1 Introduction

Ce projet consiste à développer une application en langage assembleur permettant de gérer un petit registre de personnes avec leur nom et leur âge. Le programme fonctionne dans un terminal et propose un menu interactif permettant d'effectuer différentes opérations sur les données enregistrées.

L'assembleur NASM (Netwide Assembler) a été choisi pour ce projet car il permet une compréhension approfondie du fonctionnement bas niveau d'un ordinateur, de la gestion mémoire et des appels système.

1.1 Contexte du projet

La programmation en assembleur représente le plus bas niveau d'abstraction avant le code machine. Ce projet a été réalisé dans le cadre d'un cursus universitaire en informatique pour approfondir la compréhension de l'architecture des ordinateurs et des mécanismes sous-jacents aux langages de plus haut niveau.

1.2 Objectifs

L'objectif principal du projet est de manipuler des structures de données simples et d'implémenter des fonctionnalités de gestion de base en langage assembleur :

— Lecture d'entrée utilisateur

— Stockage en mémoire

— Traitement de données

— Affichage des résultats

— Gestion des erreurs et validation des saisies

Ce projet permet également de maîtriser les concepts fondamentaux suivants :

— Architecture x86 et jeu d'instructions

— Appels système Linux (sys_read, sys_write, sys_exit)

— Manipulation de chaînes de caractères

— Gestion des registres et de la pile

1.3 Fonctionnalités implémentées

Le programme implémente six fonctionnalités principales accessibles via un menu interactif :

1. **Enregistrer une personne** : Saisie du nom et de l'âge avec validation
2. **Lister toutes les personnes** : Affichage numéroté de toutes les entrées
3. **Supprimer une personne** : Suppression par numéro avec décalage automatique
4. **Afficher les extrêmes** : Identification de la personne la plus âgée et la plus jeune
5. **Calculer l'âge moyen** : Moyenne arithmétique des âges enregistrés
6. **Quitter le programme** : Sortie propre du programme

2 Architecture du programme

2.1 Structure du programme

Le programme est structuré en deux sections principales : la section `.data` qui contient toutes les variables et les chaînes de caractères utilisées pour l'affichage, et la section `.text` qui contient le code exécuté.

2.1.1 Section des données (.data)

Cette section contient :

— Le menu affiché à l'utilisateur (6 options + invite)

— Les messages d'erreur et de confirmation (succès, échec, liste vide)

— Les invites pour les saisies (nom/âge, numéro de suppression)

— Les en-têtes pour l'affichage des résultats (liste, extrêmes, moyenne)

— Les caractères utilitaires (espace, saut de ligne, parenthèses)

— Les tableaux de stockage pour les noms et les âges

— Les buffers utilisés pour les entrées clavier et la conversion de nombres

2.1.2 Section du code (.text)

Cette section contient toutes les fonctions du programme. Le point d'entrée est `_start` qui appelle la fonction d'affichage du menu.

3 Description détaillée du code

3.1 Section .data

La section `.data` contient toutes les constantes et variables initialisées du programme :

```
1  section .data
2      menu db "1. Enregistrer du personnel",10
3           db "2. Lister des personnes enregistrees",10
4           db "3. Supprimer une personne specifique",10
5           db "4. Afficher la personne la plus agee et la plus jeune",10
6           db "5. Afficher l'age moyen",10
7           db "6. Quitter le programme",10
8           db "Votre choix: ",0
9      menu_len equ $-menu
10
11     prompt_input db "Entrez le nom et l'age (ex: Bob 25): ",0
12     success_msg db "Personne enregistree!",10,0
13     error_msg db "Entree invalide! (Nom < 20 char, 0 < age <= 120)",10,0
```

Listing 1 – Section .data - Messages et données statiques

3.2 Section .bss (implicite)

Le programme utilise une zone de mémoire non initialisée pour les buffers :

```
section .bss
    personnes resb 640      ; 10 * 64 octets
    ages resd 10            ; 10 * 4 octets
    count resd 1            ; Compteur
    input resb 256          ; Buffer d'entr e
    input_length resd 1     ; Longueur de l'entr e
    num_buffer resb 11      ; Buffer pour conversion
```

Listing 2 – Données non initialisées

3.3 Fonction display_menu

Cette fonction gère l'affichage du menu et la lecture du choix utilisateur :

```
display_menu:
    ; Effacement du buffer num rique
    mov ecx, num_buffer
    mov edx, 11
clear_num_buffer_loop:
    mov byte [ecx], 0
    inc ecx
    dec edx
    jnz clear_num_buffer_loop

    ; Affichage du menu
    mov eax, 4          ; sys_write
    mov ebx, 1          ; stdout
    mov ecx, menu
    mov edx, menu_len
    int 0x80

    ; Lecture du choix utilisateur
    mov eax, 3          ; sys_read
    mov ebx, 0          ; stdin
    mov ecx, input
    mov edx, 3          ; 3 caract res max
    int 0x80
```

Listing 3 – Fonction d'affichage du menu

3.4 Fonction parse_input

L'analyse de l'entrée utilisateur est critique. La fonction doit :

— Trouver l'espace séparateur nom/âge

— Vérifier la longueur du nom (20)

— Copier le nom dans le tableau

— Convertir la chaîne d'âge en entier

— Valider l'âge (1-120)

```
parse_input:
    mov eax, [count]
    cmp eax, 10
    jge parse_done_fail

    imul eax, 64
    lea edi, [personnes + eax]
    mov esi, input
    xor ecx, ecx

find_space_loop:
    mov al, [esi + ecx]
    cmp al, ' '
    je found_space
    cmp al, 10
    je parse_done_fail
    inc ecx
    cmp ecx, 20
    jge parse_done_fail
    cmp ecx, [input_length]
    jl find_space_loop
    jmp parse_done_fail
```

Listing 4 – Fonction d'analyse de l'entrée

3.5 Gestion de la suppression

L'implémentation de la suppression est particulièrement intéressante car elle nécessite un décalage des éléments suivants :

```
shift_loop:
    ; D calage des noms
    mov eax, esi
    dec eax
    imul eax, 64
    imul edx, esi, 64
    lea edi, [personnes + eax]
    lea ecx, [personnes + edx]

    ; Copie du nom
    push ecx
    xor edx, edx
find_shift_length:
    cmp byte [ecx + edx], 0
    je shift_length_done
    inc edx
    cmp edx, 20
    jl find_shift_length
shift_length_done:
    inc edx
    ; ... (copie effective)
```

Listing 5 – Boucle de décalage pour la suppression

4 Structures de données

Le programme utilise deux tableaux principaux pour stocker les informations :

Structure	Taille	Description
personnes	10 × 64 octets	Noms (chaînes terminées par 0)
ages	10 × 4 octets	Âges (entiers 32 bits)
count	4 octets	Compteur de personnes
input	256 octets	Buffer d'entrée utilisateur
input_length	4 octets	Longueur de l'entrée lue
num_buffer	11 octets	Buffer de conversion numérique

TABLE 1 – Structures de données utilisées

Une variable `count` est utilisée pour suivre le nombre actuel de personnes enregistrées. Cette variable est incrémentée lors de l'ajout et décrémentée lors de la suppression.

5 Gestion des erreurs

Le programme inclut plusieurs vérifications pour garantir l'intégrité des données :

5.1 Validation des entrées

— **Choix du menu** : Vérification que l'utilisateur entre un seul chiffre valide (1-6)
— **Nom** : Ne doit pas dépasser 20 caractères et ne peut pas être vide
— **Âge** : Doit être compris entre 1 et 120 (inclus)
— **Format** : La saisie doit contenir un nom et un âge séparés par un espace

5.2 Gestion des limites

— Le programme empêche l'ajout de plus de 10 personnes (taille fixe des tableaux)
— La suppression vérifie que le numéro entré correspond à une personne existante
— L'affichage et les calculs sont adaptés si la liste est vide

5.3 Messages d'erreur

Les entrées invalides affichent un message d'erreur explicite :
— "Entree invalide ! (Nom < 20 char, 0 < age <= 120)"
— "Cette personne n'existe pas !"
— "Aucune personne n'est encore enregistree !"
— "Precisez votre choix avec un seul numero !"

6 Exemples d'exécution

6.1 Ajout d'une personne

```
1. Enregistrer du personnel
2. Lister des personnes enregistrees
3. Supprimer une personne specifique
4. Afficher la personne la plus agee et la plus jeune
5. Afficher l'age moyen
6. Quitter le programme
Votre choix: 1
Entrez le nom et l'age (ex: Bob 25): Alice 30
Personne enregistree!
```

6.2 Affichage de la liste

```
Votre choix: 2
Liste des personnes:
1 Alice 30
2 Bob 25
3 Charlie 35
```

6.3 Calcul de la moyenne

```
Votre choix: 5
Age en moyenne:
30 (30 25 35)
```

7 Tests et validation

7.1 Plan de tests

Test	**Entrée**	**Résultat attendu**
Enregistrement normal	"Bob 25"	Succès
Nom trop long	"ABCDEFGHIJKLMNOPQRSTU 30"	Erreur
Âge invalide	"Bob 150"	Erreur
Âge non numérique	"Bob abc"	Erreur
Suppression valide	"2"	Succès
Suppression invalide	"99"	Erreur
Liste vide	-	Message approprié
10 personnes max	11e entrée	Erreur

Table 2 – Cas de test validés

7.2 Résultats des tests

Tous les tests ont été exécutés avec succès. Le programme gère correctement :

— Les cas nominaux
— Les erreurs de saisie
— Les limites des structures
— Les cas particuliers (liste vide, suppression, etc.)

8 Difficultés rencontrées et solutions

8.1 Difficultés techniques

— **Manipulation des chaînes** : En assembleur, il n'existe pas de fonctions natives pour manipuler les chaînes. Nous avons dû implémenter manuellement la copie, la recherche de caractères et la terminaison par zéro.
— **Conversion chaîne entier** : La fonction `convert_age` a nécessité une implémentation précise avec multiplication par 10 et gestion des caractères non numériques.
— **Décalage mémoire pour la suppression** : La suppression d'un élément nécessite de décaler tous les éléments suivants, en préservant l'intégrité des données. Cela a été complexe à implémenter correctement.
— **Gestion des registres** : Il a fallu être très vigilant pour ne pas écraser accidentellement des valeurs importantes dans les registres entre les appels de fonction.

8.2 Solutions apportées

— Utilisation de boucles explicites pour toutes les manipulations de chaînes
— Sauvegarde systématique des registres sur la pile avant les opérations critiques
— Validation progressive des entrées pour détecter les erreurs au plus tôt
— Commentaires détaillés pour documenter l'utilisation des registres
— Tests exhaustifs de tous les cas limites

9 Améliorations possibles

9.1 Améliorations fonctionnelles

— **Recherche par nom** : Ajouter une fonctionnalité de recherche
— **Modification** : Permettre de modifier une entrée existante
— **Sauvegarde persistante** : Enregistrer les données dans un fichier
— **Tri** : Trier la liste par nom ou par âge
— **Statistiques avancées** : Médiane, écart-type, etc.
— **Export** : Exporter les données au format CSV

9.2 Améliorations techniques

— **Macros** : Créer des macros pour les appels système fréquents

— **Gestion dynamique** : Allocation dynamique de mémoire pour un nombre illimité de personnes

— **Optimisation** : Optimiser les boucles de copie avec des instructions SIMD

— **Interface améliorée** : Utilisation de ncurses pour une interface plus conviviale

9.3 Exemple d'amélioration : Macro pour sys_write

```
%macro PRINT 2
    mov eax, 4
    mov ebx, 1
    mov ecx, %1
    mov edx, %2
    int 0x80
%endmacro

; Utilisation simplifiee :
; PRINT menu, menu_len
```

Listing 6 – Macro proposée pour simplifier les écritures

10 Compilation et exécution

10.1 Commandes de compilation

Le programme peut être compilé et lié avec les commandes suivantes :

```
; Assemblage du code source
nasm -f elf32 -o projet.o projet.asm

; Edition de liens pour cr er l'ex cutable
ld -m elf_i386 -o projet projet.o

; Ex cution du programme
./projet
```

Listing 7 – Compilation et édition de liens

10.2 Prérequis

Pour compiler et exécuter ce programme, vous avez besoin de :

— NASM (Netwide Assembler)

— Un système Linux 32 bits ou les bibliothèques 32 bits sur un système 64 bits

— Les outils de développement de base (binutils pour ld)

11 Conclusion

Ce projet démontre l'utilisation du langage assembleur pour créer une application interactive simple mais complète. Il met en pratique :

— La gestion de la mémoire avec des tableaux statiques

— Les appels système Linux (sys_read, sys_write, sys_exit)

— La manipulation de chaînes de caractères au niveau le plus bas

— La validation des entrées utilisateur

— La gestion d'erreurs et de cas limites

— L'implémentation d'algorithmes simples (recherche de min/max, calcul de moyenne)

Malgré sa simplicité apparente, ce projet illustre des concepts fondamentaux de la programmation bas niveau et permet de mieux comprendre le fonctionnement interne des ordinateurs. La programmation en assembleur, bien que plus complexe que les langages haut niveau, offre un contrôle total sur le matériel et une compréhension approfondie des mécanismes sous-jacents.

Le programme est fonctionnel, robuste et répond à tous les objectifs fixés initialement. Il constitue une base solide pour d'éventuelles extensions ou améliorations futures.

— Fin du rapport —